

**Project Title:**

**Room-Based Chat App**

# Introduction

The **Room-Based Chat App** is a web-based communication platform that allows users to engage in real-time text-based conversations within different chat rooms. The application enables users to create rooms, join existing ones, and chat with others in a secure and efficient manner. It integrates modern web technologies to provide a dynamic and user-friendly experience.

## Project Objectives

- **Real-Time Communication:** Enable users to communicate instantly through messages in chat rooms.
- **User Management:** Provide authentication and personalized user experiences.
- **Room Management:** Allow the creation and management of multiple chat rooms, each with unique settings.
- **Data Persistence:** Store chat messages and room information persistently.
- **Scalability and Security:** Build a system that scales efficiently and ensures data privacy and security.

## Key Features

- **User Authentication & Authorization:** Users can sign up, log in, and securely access different chat rooms.
- **Room Creation:** Users can create new chat rooms, choosing visibility options like public or private rooms.
- **Real-Time Messaging:** Through WebSockets, users can send and receive messages instantaneously in rooms.
- **Room Management:** Admins can moderate and manage chat rooms, control user access, and delete or mute messages.
- **Message History:** All messages are stored in the PostgreSQL database, allowing users to access past conversations.
- **Multi-User Support:** Multiple users can participate in the same chat room simultaneously, making the platform suitable for group chats and discussions.
- **User Profiles:** Users have individual profiles linked to their accounts, and they can customize their username and avatar.

- **PostgreSQL Integration:** A robust relational database stores user, room, and message data efficiently.

## Technology Stack

- **Frontend:**

- **React.js:** A powerful library for building responsive and interactive user interfaces.
- **Vite:** A next-generation build tool that enables fast development with features like hot module replacement (HMR).
- **React Router:** For navigation between different views like login, chat rooms, and messages.
- **WebSocket Client:** For enabling real-time communication between users in different chat rooms.

- **Backend:**

- **Spring Boot:** A Java-based framework that simplifies the development of REST APIs and web applications. It serves as the backend for the chat app.
- **Spring Security:** Used for implementing secure authentication and authorization mechanisms.
- **Spring Data JPA:** For simplifying database interaction and managing entities like Users, Rooms, and Messages.
- **WebSocket Protocol:** For implementing real-time bidirectional communication between the server and clients.

- **Database:**

- **PostgreSQL:** A relational database management system used to store all data related to users, rooms, and messages. It ensures data consistency and integrity.

## System Architecture

The system follows a **Client-Server** architecture, where:

**Frontend (Client)** communicates with:

- The **Backend (Server)** to handle user authentication, room creation, and message management via RESTful APIs.

- **WebSocket connections** to send and receive messages in real-time.

**Backend (Spring Boot)** serves:

- RESTful APIs for user authentication, chat room creation, and retrieval of messages.
- WebSocket endpoints for enabling real-time message communication.
- **PostgreSQL Database** for storing persistent data related to users, rooms, and messages.

## Detailed Workflow

### • User Interaction:

- **User Registration/Login:** New users sign up with their details, and returning users log in with their credentials. Secure login is facilitated via **JWT** (JSON Web Tokens).
- **Room Interaction:**
  - Users can browse, join, or create chat rooms.
  - Rooms have visibility settings, allowing for both public and private rooms.
  - Users can send messages in real time within any active room they are part of.

### • Real-Time Communication:

- WebSocket connections are established when a user enters a chat room.
- Messages typed by users are sent to the backend, which then broadcasts them to all users in the room in real-time.
- Every user in the room receives messages without the need to refresh the page.

### • Data Persistence:

- **Message Persistence:** Every chat message is stored in the PostgreSQL database, allowing users to access past messages even after they leave or return to the room.
- **Room and User Data:** All relevant data, such as room configurations and user profiles, are securely stored in the database.

# Security Features

- **Authentication & Authorization:**
  - **JWT** is used for secure authentication, ensuring that users' sessions are protected. Only authorized users can access private rooms and send messages.
  - **Spring Security** ensures that users are properly authenticated before they can interact with the system.
- **Data Protection:**
  - WebSocket connections are secured, and all messages are transmitted over **HTTPS** to prevent eavesdropping.
  - **SQL Injection Prevention:** Spring Data JPA abstracts direct database access, minimizing the risk of SQL injection attacks.
- **Room and Message Moderation:**
  - Admins can block users or delete messages, ensuring that inappropriate content is managed within rooms.

# Database Design

- The **PostgreSQL** database consists of three main entities:
- **Users:** Stores user information such as usernames, emails, and password hashes.
- **Rooms:** Contains data about each chat room (room name, privacy settings, and admin details).
- **Messages:** Stores the content of messages, the associated user, room ID, and timestamp.
- Relationships:
- A **User** can be a member of multiple **Rooms**.
- A **Room** can have multiple **Messages**, each associated with a **User**.

# Performance and Scalability

- **Scalability:** The app is built to scale horizontally, allowing more chat rooms and users to be supported as the system grows.
- **Efficiency:** Vite ensures fast development builds, while React optimizes rendering performance with its virtual DOM.

- **WebSocket Efficiency:** Real-time communication is managed through WebSockets, reducing the overhead compared to traditional polling methods.

## Challenges Faced

- **Real-Time Messaging:** Ensuring that messages were sent and received instantly across all users in a room required efficient use of WebSockets and proper server handling.
- **Database Performance:** Managing a large number of messages and room data in PostgreSQL required careful attention to indexing and query optimization to ensure scalability.
- **User Authentication:** Implementing secure JWT-based authentication for both new users and returning ones was crucial for data protection.

## Future Enhancements

- **Voice/Video Integration:** Future versions of the app could include voice and video chat features for a more interactive communication experience.
- **File Sharing:** Users could upload and share files such as images, documents, and other media types within rooms.
- **Mobile Support:** A mobile-friendly version of the app can be developed to ensure users have a seamless experience across devices.

## Conclusion

- The **Room-Based Chat App** successfully combines real-time messaging, user management, and room creation functionalities, offering a user-friendly platform for online communication. By using **React**, **Spring Boot**, and **PostgreSQL**, the app provides a highly scalable, efficient, and secure solution for users to interact in various chat rooms. The project serves as an excellent example of integrating modern web technologies to build a robust and feature-rich communication application.